RAG-Based Customer Support Assistant (LangGraph + HITL)

Objective

The objective of this project is to design a RAG-based Customer Support Assistant that can answer user queries using information from a knowledge base. The system retrieves relevant data using embeddings and generates accurate responses.

It uses LangGraph for workflow control and supports Human-in-the-Loop (HITL) to handle queries when no relevant answer is found.

1. System Overview

Problem Definition

Traditional customer support systems rely heavily on human agents, leading to delays, inconsistent responses, and increased operational costs. Users often face difficulty in getting quick and accurate answers for common queries such as refunds, contact details, and password reset.

Proposed Solution

This system implements a **Retrieval-Augmented Generation (RAG)** based assistant that:

- Retrieves relevant information from a knowledge base
- Generates contextual responses
- Uses workflow-based decision-making (LangGraph)
- Escalates complex queries to humans (HITL)

Scope of the System

- Handles FAQs from a knowledge base (PDF/TXT)
- Supports queries like refund, contact, password reset
- Provides instant responses
- Escalates unknown queries to human support
- Designed for web-based interaction using Streamlit

2. Architecture Diagram

2.1 Overview

The system follows a layered architecture combining Retrieval-Augmented Generation (RAG), workflow orchestration using LangGraph, and Human-in-the-Loop (HITL) escalation.

It consists of the following major components:

- User Interface
- Document Ingestion Pipeline

- Embedding System
- Vector Database
- Retrieval Layer
- LLM Processing Layer
- Workflow Orchestration (LangGraph)
- HITL System

2.2 Architecture Components

1. User Interface (CLI / Web)

- Implemented using Streamlit Web UI
- Allows users to:
    - Enter queries
    - View responses
- Acts as the entry point to the system

2. Document Ingestion Pipeline (Offline Phase)

This pipeline prepares the knowledge base for retrieval.

Includes:

- **Document Loader**
    - Loads PDF/TXT files
- **Chunking**
    - Splits document into smaller pieces
- **Preprocessing**
    - Cleans and formats text

3. Embedding System

- Uses HuggingFace Embeddings (MiniLM)
- Converts text chunks into vector representations
- Enables semantic search instead of keyword matching

4. Vector Database (ChromaDB)

- Stores embeddings of all document chunks
- Supports fast similarity search
- Efficient for retrieval-based systems

5. Retrieval Layer

- Fetches top-k relevant chunks from vector database
- Filters results based on query relevance
- Ensures only useful context is passed forward

6. LLM Processing Layer

- Uses retrieved context to generate answers
- Produces human-readable responses
- Improves accuracy using RAG approach

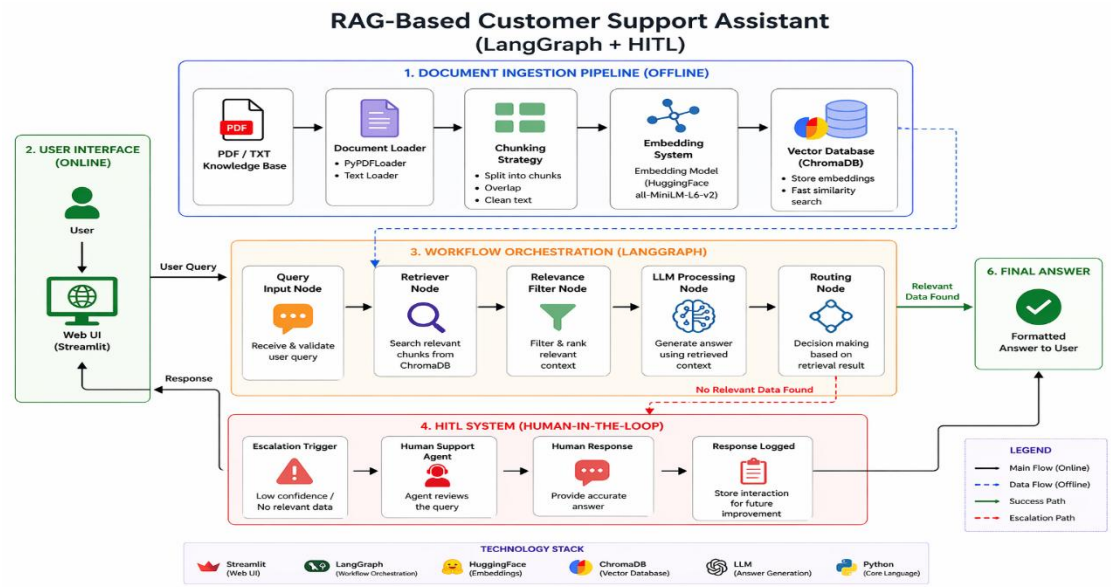
7. Workflow Orchestration (LangGraph)

- Controls system execution flow
- Implements:
    - Input → Process → Output flow
- Handles:
    - Query processing
    - Decision-making
    - Routing logic

8. HITL System (Human-in-the-Loop)

- Activated when:
    - No relevant answer found
    - Query is complex
- Routes query to human support agent
- Ensures system reliability

Flow Summary



3. Component Description

Document Loader

Loads the knowledge base (PDF/TXT).
Currently uses TXT for simplicity, but extendable to PDF using PyPDFLoader.

Chunking Strategy

- Splits document into smaller pieces (lines/chunks)
- Improves retrieval accuracy
- Reduces irrelevant context

Embedding Model

- Uses **HuggingFace (MiniLM model)**
- Converts text into vector embeddings
- Enables semantic similarity sear

Vector Store (ChromaDB)

- Stores embeddings efficiently
- Supports fast similarity-based retrieval
- Lightweight and easy to integrate

Retriever

- Fetches top-k relevant chunks
- Filters results based on query relevance
- Returns best matching content

LLM (Language Model)

- Generates responses using retrieved content
- Ensures context-aware answers

Graph Workflow Engine (LangGraph)

- Controls flow of execution
- Implements **Input → Process → Output**
- Enables conditional routing

Routing Layer

- Decides system behavior based on query:
  - Relevant → Answer generation
  - Irrelevant → HITL escalation

HITL Module (Human-in-the-Loop)

- Triggered when:
  - No relevant documents found
  - Query is complex
- Redirects to human support

4. Data Flow

4.1 Document Processing Flow (Offline Phase)

The system processes the knowledge base before handling user queries.

Steps:

1. The knowledge base is provided as a **PDF/TXT document**
2. The document is loaded using a **Document Loader**
3. Content is divided into smaller chunks (chunking)
4. Each chunk is converted into vector embeddings
5. Embeddings are stored in **ChromaDB vector database**

Flow Representation:

```
PDF/TXT → Load → Chunk → Embed → Store in ChromaDB
```

4.2 Query Lifecycle (Online Phase)

When a user asks a question, the system processes it as follows:

Steps:

1. User enters query via Streamlit UI

2. Query is passed to the **LangGraph workflow**
3. Retriever searches for relevant chunks from ChromaDB
4. Retrieved results are filtered for relevance
5. System decides:
    - If relevant data found → Generate answer
    - If no relevant data → Trigger HITL
6. Final response is returned to user

Flow Representation:

```
User Query
   ↓
Retriever (ChromaDB Search)
   ↓
Filter Relevant Content
   ↓
[Decision Layer]
   → If Found → Generate Answer
   → If Not Found → HITL Escalation
   ↓s
Final Response to User
```

5. Technology Choices

5.1 ChromaDB (Vector Database)

Why ChromaDB?

- Lightweight and easy to set up locally
- Fast similarity search using embeddings
- No complex configuration required
- Suitable for small to medium-scale applications

5.2 LangGraph (Workflow Engine)

Why LangGraph?

- Enables graph-based execution flow
- Supports conditional routing (important for HITL)
- Clearly separates processing logic into nodes
- Ideal for decision-making systems

5.3 LLM Choice

Chosen Approach:

- Lightweight LLM integrated with retrieval

Reasons:

- Faster response time
- Reduced cost compared to large models
- Works efficiently with RAG (context-driven answers)

5.4 Additional Tools

Tool	Purpose
Streamlit	Web-based user interface
LangChain	Integration framework
HuggingFace Embeddings	Convert text to vectors
Python	Core implementation

6. Scalability Considerations

6.1 Handling Large Documents

- Chunking reduces memory load
- Only relevant chunks are retrieved
- Vector storage allows efficient indexing

6.2 Increasing Query Load

- Retrieval from ChromaDB is fast
- System can be scaled using:
    - Cloud deployment
    - Parallel query processing
- Stateless query handling improves scalability

6.3 Latency Concerns

- Precomputed embeddings reduce processing time
- Only top-k relevant chunks are retrieved
- Lightweight models improve response speed